Sensiq: A Serverless IoT Architecture for Cost-Effective Laboratory Monitoring

Technical Reports: CL-2026-42, July 2026

Simon Völkl, Johannes Schieder, Sebastian Rosner, Andre Tien Vu and Christoph P. Neumann
Department of Electrical Engineering, Media, and Computer Science
Ostbayerische Technische Hochschule Amberg-Weiden
Amberg, Germany

***Abstract*—Background: Reliable indoor environment quality (IEQ) monitoring is critical for safety and experimental validity in laboratories. Existing Internet of Things (IoT) solutions often rely on costly on-premises servers, coarse sampling intervals, or proprietary interfaces.**

Objective: This paper introduces Sensiq, a low-latency, cost-efficient, and fully serverless end-to-end IoT architecture designed to continuously track environmental parameters such as temperature, humidity, gas concentration, and flame exposure.

Methods: The system leverages low-cost ESP32 microcontroller edge nodes equipped with multiple sensors to acquire and aggregate data. Measurements are securely transmitted via Message Queuing Telemetry Transport (MQTT) to an Amazon Web Services (AWS) serverless backend. The cloud architecture dynamically splits data into two optimized paths: a live route for sub-second real-time state retrieval, and a history route for scalable long-term analytics. A platform-independent React dashboard provides interactive visualization, supplemented by email notifications for critical threshold breaches.

Results: The Sensiq system confirms that a reliable, low-cost monitoring system is entirely feasible. Ultimately, this implementation serves as a practical example that continuous, high-frequency data acquisition can be achieved extremely cost-efficiently, requiring roughly 43 € for hardware prototyping and approximately 1.87 € in monthly operational overhead per node.

Conclusion: Sensiq provides a highly scalable, maintenance-free solution for continuous laboratory supervision. Future enhancements will prioritize security, analytics, deployment flexibility, threshold configurability and advanced predictive capabilities using machine learning models.

***Index Terms*—Internet of Things (IoT), Environmental Monitoring, Serverless Architecture, Amazon Web Services (AWS), ESP32, Indoor Air Quality (IAQ), Cloud Computing**

I. Introduction

Reliable indoor environment quality (IEQ) monitoring is a recurring requirement in laboratories, production halls, and similar facilities, where ambient conditions like indoor air quality (IAQ) directly affect experimental validity, product quality, and occupational safety. Existing Internet of Things (IoT) monitoring platforms address parts of this problem but typically rely on on-premises servers, offer only coarse sampling, or restrict access through proprietary applications (Section II).

This report describes Sensiq, a microcontroller unit (MCU) based IoT architecture combining ESP32 (Espressif Systems) edge devices with a serverless Amazon Web Services (AWS) cloud backend. Sensor readings are transmitted securely over Message Queuing Telemetry Transport (MQTT) using Transport Layer Security (TLS), validated, and persisted for both live and historical views. Exposed through Representational State Transfer (REST) Application Programming Interface (API) endpoints, a React-based frontend visualises the data, while threshold-driven email alerts notify personnel of critical events.

A. Mission Statement

Sensiq aims to deliver an end-to-end, low-latency environmental monitoring system that provides both real-time visibility and cost-efficient historical analytics. By leveraging a fully serverless backend, it enables stakeholders to continuously track critical ambient parameters—such as temperature, humidity, light intensity, gas concentration, and flame exposure—to ensure safety and quality assurance.

B. Motivation

Unnoticed deviations in ambient conditions can invalidate experiments, damage sensitive samples, and endanger personnel [1, 2]. While continuous monitoring is essential, existing academic IoT platforms (e.g., [3, 4]) often rely on high-maintenance on-premises stacks and employ sampling intervals that are too coarse to capture short-lived events. Sensiq closes this gap by pairing inexpensive edge hardware with a scalable cloud architecture, keeping infrastructure and operating costs low enough for routine laboratory use. The full source code is publicly available under the MIT License [5].

C. Document Structure

The remainder of this report is organised as follows. Section II positions Sensiq against existing academic and commercial monitoring solutions. Section III introduces the conceptual building blocks of the system.

The subsequent three sections refine this concept along the data flow: Section IV details the ESP32-based sensor node and its secure MQTT publication to AWS IoT Core; Section V describes the serverless cloud pipeline, covering both the low-latency live route and the long-term history route; and Section VI explains how end users consume the data through the React-based web frontend and how they are notified of critical events via email. Section VII assesses the resulting system with respect to testability, operating cost, and architectural trade-offs. Finally, Section VIII summarises the contribution of the report and outlines directions for future work.

Please cite as: Simon Völkl, Johannes Schieder, Sebastian Rosner, Andre Tien Vu, and Christoph P. Neumann. *Sensiq: A Serverless IoT Architecture for Cost-Effective Laboratory Monitoring*. Tech. rep. CL-2026-42. Ostbayerische Technische Hochschule Amberg-Weiden, Department of Electrical Engineering, Media, and Computer Science, July 2026.

II. Related Work

Several IoT-based platforms for indoor environmental monitoring have been proposed. This section contrasts two representative academic systems — by Mumtaz et al. [3] and Marques and Pitarma [4] — with the design decisions taken for Sensiq.

A. On-Premises IoT Air Quality Monitoring

Mumtaz et al. [3] present a multi-sensor node targeting air quality prediction. While sharing Sensiq's general topology, their system differs fundamentally in three areas.

First, they rely on a local web server and database, tying the platform to on-premises operation. Sensiq instead targets a fully serverless cloud backend (Section V) to scale on demand and keep idle costs near zero.

Second, their five-minute sampling interval is geared towards trend analysis, which is too coarse for safety-relevant events like fires or for product quality assurance where short-lived deviations can compromise samples. Sensiq utilizes a low-latency MQTT-over-TLS pipeline to surface critical conditions immediately.

Third, their twenty-hour battery autonomy limits remote deployment. Sensiq's ESP32 architecture could support deep sleep operation, as mentioned in Section VII.

B. iAQ+ Laboratory Environment Supervision System

Marques and Pitarma [4] propose iAQ+, an ESP8266-based monitoring system. While hardware choices are similar, significant differences exist in backend architecture, frontend access, and timing guarantees.

On the backend, iAQ+ employs a traditional server-based stack (SQL Server and .NET), which requires continuous hosting. Sensiq delegates persistence and routing to managed AWS services.

On the frontend, iAQ+ restricts access to iOS devices and limits the web dashboard to hourly aggregations. Sensiq provides platform-independent access via open REST endpoints, a React frontend, and a true real-time view (Section VI).

Finally, iAQ+ lacks explicit latency guarantees for threshold alerts. Sensiq is efficiently designed around instantaneous, threshold-driven email notifications.

III. Technical Concept

The Sensiq system comprises three core logical tiers: the physical edge tier for data acquisition, the cloud backend for processing and storage, and the presentation tier for user interaction. Figure 1 illustrates the overarching system architecture.

At the edge, microcontroller-based sensor nodes acquire environmental metrics. These measurements are aggregated and transmitted securely to the cloud infrastructure via MQTT. The data stream is subsequently split into two parallel processing branches:

- **Live Route:** A low-latency path that validates, evaluates, and caches the most recent device state for real-time visualization and alerting.
- **History Route:** A long-term storage path that buffers and transforms telemetry data into a column-oriented format, enabling optimized historical queries.

Clients interact with the system through a web-based dashboard, which fetches data from the backend via a REST API, and through automated email notifications triggered by critical environmental thresholds.

IV. Hardware

The sensor node is the physical edge component of the Sensiq system, built around an ESP32 microcontroller with a set of environmental sensors. This section maps the hardware components (Section IV-A), outlines the firmware measurement loops (Section IV-B), and details the secure MQTT publication strategy (Section IV-C).

A. Component Overview

The sensor node is built using the following core components:

- **ESP32 NodeMCU:** Central microcontroller with integrated Wi-Fi and Bluetooth [6, 7].
- **DHT11:** Basic digital temperature and humidity sensor [8].
- **Joy-it KY-026 & KY-028:** Analog flame sensor (IR band) and NTC thermistor, both with adjustable digital thresholds [9, 10].
- **TSL2561:** Digital ambient-light sensor for visible and IR light [11, 12].
- **Bosch BME680:** Comprehensive sensor for precise temperature, humidity, pressure, and VOC gas concentrations [13–15].
- **Auxiliary Switches:** Two inputs reserved for labeling training data and outliers (see Section VIII).

The hardware design is documented using two complementary views. Figure 2 shows the logical circuit diagram (created with KiCAD [16]), detailing wiring and GPIO assignments. The functional prototype is shown in Figure 3, demonstrating the assembled sensor node.

B. Measurement Workflow

The firmware on the ESP32 follows a deterministic, two-phase lifecycle written in Arduino C++ [17]: a one-time *startup* phase that prepares the networking and sensing subsystems, followed by a continuously running *measurement loop*.

During startup the node performs the following steps in order:

1) Wi-Fi association: The node connects to a configured local network using a pre-provisioned SSID and password.
2) Time synchronization: The internal clock is synchronized with an NTP server. An accurate wall-clock time is required both for meaningful measurement timestamps and for the validation of TLS certificates during the subsequent MQTT handshake.
3) Sensor initialization: GPIO pin modes are set and sensor-specific configuration (e.g. DHT11 configuration) is applied.

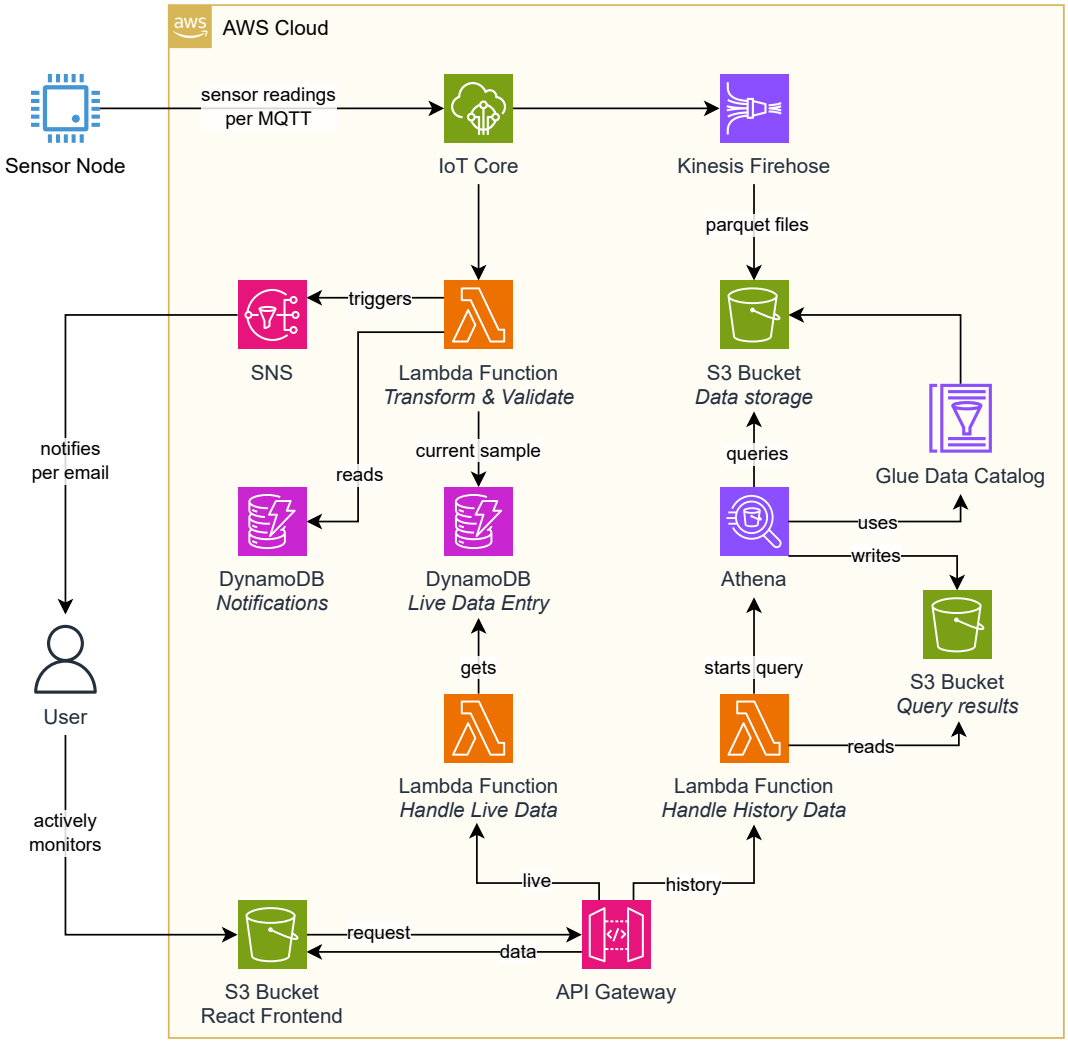

Figure 1. Cloud processing architecture depicting the sensor node, backend services, and user interaction components.

4) MQTT setup: The MQTT client is configured (see Section IV-C).

Once startup is complete, the node enters its measurement loop:

- The MQTT connection is continuously validated and kept alive; on disconnect, the client transparently reconnects.
- Every t milliseconds a full sensor reading is taken across all attached sensors with a default waiting period of $t = 1\,000\,\mathrm{ms}$. Values smaller than one second might not change much, as the sensors often have a minimum sampling rate of $1\,\mathrm{Hz}$ or lower.
- After n consecutive readings the buffered samples are aggregated into a single record, serialized as JSON and published to the configured MQTT topic. The standard sample aggregation size is $n = 5$, guaranteeing a balance between responsiveness and noise reduction.

Aggregation reduces sensor noise and the influence of isolated outliers. It is performed per field according to its data type: the most recent timestamp of the buffer is taken as the record timestamp, numerical values (analog readings) are averaged arithmetically, and digital boolean signals are decided by majority voting over the buffered samples.

For the KY-028 module, the raw 12-bit ADC reading is first converted to a resistance and then mapped to an absolute temperature in degrees Celsius using the Steinhart–Hart equation [18].

C. MQTT Publication to AWS IoT Core

The sensor node communicates with AWS IoT Core via the MQTT protocol over TLS.

To establish a secure connection to the AWS IoT broker on port 8883, the node requires the broker address and a set of cryptographic credentials. The publicly available AWS Root CA certificate alongside the individual device certificate and device private key are used for mutual TLS authentication.

The MQTT topic structure is designed around a unique device identifier (e.g., `esp32-lab-001`). Data exchange uses

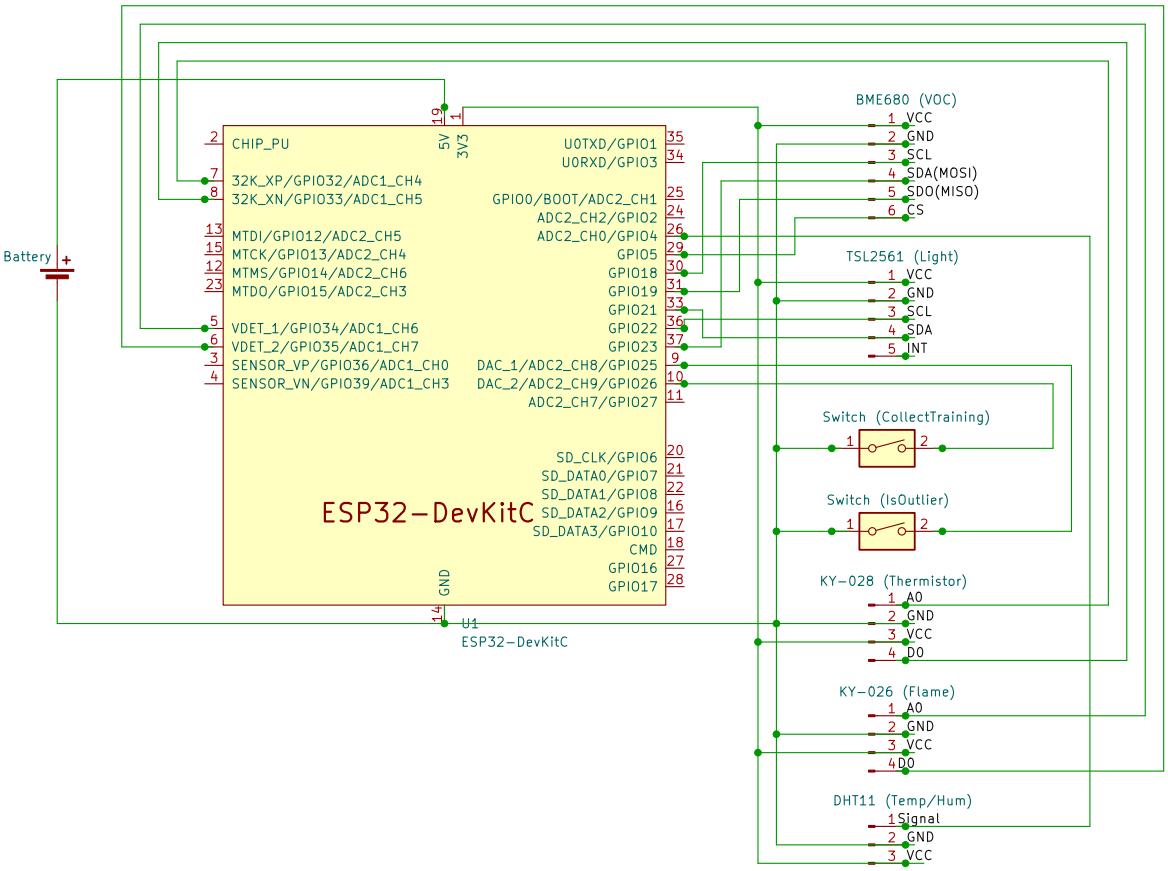

Figure 2. Circuit diagram of the sensor node, showing the logical wiring between the ESP32 microcontroller, sensors, switches and power supply, together with their respective pin assignments.

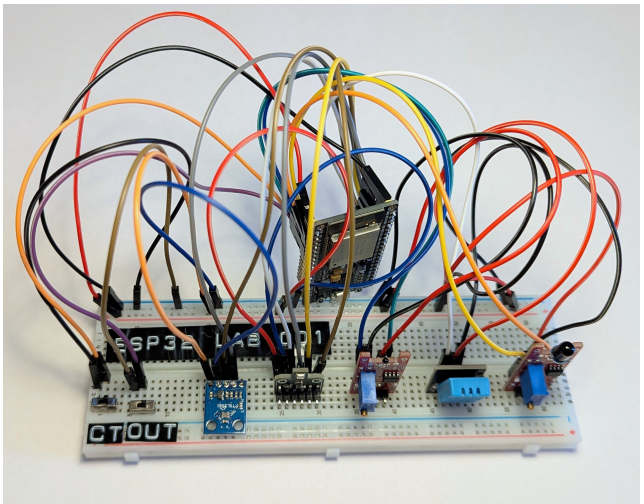

Figure 3. Real physical sensor node with the assembled sensors, the ESP32 microcontroller and the prototyping breadboard.

three primary topics: `sensiq/<identifier>/data` for publishing the aggregated sensor records, `sensiq/<identifier>/test` for verifying connectivity, and `sensiq/<identifier>/commands` reserved for receiving subsequent control commands. Listing 1 shows an example of the JSON payload that the ESP32 node publishes to the data topic after each aggregation interval.

V. AWS Services

All AWS resources are provisioned and managed using the AWS Cloud Development Kit (CDK) [19], which enables infrastructure as code with TypeScript.

A. Live Route

The live route provides a low-latency data delivery mechanism, exposing a `POST /live` endpoint via API Gateway. It returns the most recent sensor reading as a structured JSON payload. The pipeline spans from hardware ingestion through serverless transformation to a managed NoSQL store.

1) Data Ingestion and Transformation: Sensor readings are assembled on the ESP-32 microcontroller and published via MQTT to AWS IoT Core [20]. An IoT topic rule forwards each incoming message to the `handleTransformValidate` Lambda function written in Python. This function is responsible for validating and transforming the payload to the internal data

```
{
    "running_time":131147,
    "timestamp":"2026-06-15T10:45:16Z",
    "device_id":"esp32-lab-001",
    "location":"Lab A, OTH Amberg-Weiden, 92224 Amberg,
        Germany",
    "dht_humidity":61.6,
    "dht_temperature":22.6,
    "dht_heat_index":22.52377,
    "flame_analog":1089,
    "flame_digital":false,
    "thermistor_analog":2178,
    "thermistor_digital":false,
    "thermistor_temp":27.95632,
    "bme_heated_up":false,
    "bme_temperature":24.89887,
    "bme_humidity":59.40022,
    "bme_pressure":964.558,
    "bme_altitude":413.5006,
    "bme_voc":97.3798,
    "tsl_lux":167.8,
    "is_outlier":false,
    "collect_training":false
}
```


Listing 1. Published JSON payload by the ESP32 sensor.

format, and classifies outlier readings against predefined thresholds, as described in Section VI-A. Regardless of classification, the validated record is written to a DynamoDB table [21] (`LiveDataDB`) via a `PutItem` operation, which always reflects the latest device state.

2) Live Data Retrieval: Incoming `POST /live` requests are routed through the API Gateway to the `handleLiveData` Lambda function. This function performs a read against `LiveDataDB` and returns the latest sensor record as a JSON response. Because the data is served directly from DynamoDB with single-digit millisecond read latency, the live route consistently responds well within the 29-second API Gateway timeout, typically in under one second. The handler returns an error response when the entry in `LiveDataDB` is older than 60 seconds.

B. History Route

The history route provides a long-term data retrieval mechanism, exposing a `POST /history` endpoint via API Gateway. It accepts query parameters such as time ranges, limits, aggregation precision and a device ID, and returns precision-adjusted aggregated historical sensor data as a structured JSON payload. The pipeline bridges data ingestion from AWS IoT Core to a highly optimized query backend.

1) Data Ingestion and Storage: Incoming JSON messages from AWS IoT Core are captured by an IoT rule and forwarded to Amazon Kinesis Data Firehose [22]. To mitigate the small-files problem, Firehose buffers the data for up to 15 minutes or until a size of 128 MB is reached before converting it to Apache Parquet format. The Parquet files are written to a S3 Standard data bucket [23], partitioned by year, month, and day to ensure efficient storage and rapid retrieval. S3 lifecycle management policies [24] are configured to retain the data securely for an extended period, while an AWS Glue Data Catalog maintains the structural metadata of the `sensor_data` table. IAM policies tightly control access to the Glue catalog and underlying S3 buckets [25].

2) Query Execution and Processing: A dedicated AWS Lambda function coordinates the fulfillment of history requests. It parses the incoming query parameters, validates them, and constructs an Athena query using query execution parameters [26] to defend against SQL attacks. Athena utilizes Glue's partition projection mapping [27] to prune directories rapidly without needing to load external metadata. A dedicated Athena workgroup [28] manages the query execution, saving the result datasets to a secondary S3 query results bucket.

During execution, the Lambda function continually polls the query execution status [29]. Once the query yields a terminal state, the Lambda handler retrieves the generated results from the S3 bucket, formats them into a JSON response, and returns them to the API Gateway. Although Athena queries induce higher latency compared to the live route, processing is typically completed well within the 29-second timeout limit, mostly in under 5 seconds.

VI. User Interaction

Passive, push-based email notifications (Section VI-A) and an interactive web frontend (Section VI-B) offer users comprehensive capabilities for monitoring the laboratory's environmental conditions.

A. Email Notifications via SNS

Users are alerted to critical environmental conditions via automated email notifications. Threshold evaluations are executed within the `handleTransformValidate` Lambda function, which compares incoming telemetry data against predefined operational limits. To prevent alert fatigue, the system implements a rate-limiting mechanism using a DynamoDB [21] table named `EmailDB`. This table records the timestamp and trigger condition of the most recently dispatched alert. Subsequent emails are only transmitted if the previous alert occurred at least 300 seconds prior. AWS SNS [30] manages the delivery of these notifications to a curated list of subscriber addresses. Each email provides a detailed summary of the device's current state and identifies the specific metric that triggered the alarm.

B. Web Frontend

The frontend consists of a single-page React application hosted on AWS S3. It integrates with the backend infrastructure exclusively via the REST API endpoints detailed in Section V. Specifically, the `POST /live` route supplies data to real-time status tiles and individual sensor indicators, reflecting the most recent device telemetry. Conversely, the `POST /history` route populates historical data charts, allowing users to interactively adjust the displayed time ranges and metrics. Currently, access to the

Figure 4. React Frontend with real-time status tiles and a historical data chart.

dashboard is unrestricted, rendering it publicly accessible. A representative illustration of the user interface is provided in Figure 4.

VII. Evaluation

A comprehensive evaluation of the testability, cost estimation, and architectural trade-offs is presented in the following subsections.

A. Testability

The cloud system's testability is ensured through rigorous automated checks for both infrastructure and application logic. Local `yarn` scripts automate test execution and coverage reporting.

1) AWS Infrastructure: The AWS cloud infrastructure configuration is automatically validated using built-in AWS Cloud Development Kit (CDK) assertions, leveraging the `Template` and `Match` constructs. This ensures that the generated resources and policies match the intended architectural design prior to deployment.

2) AWS Lambda Handlers: AWS Lambda functions are thoroughly unit-tested using Python's `unittest` or `pytest` frameworks, employing exhaustive mocking to isolate execution behavior. To evaluate Lambda handlers independently of the API Gateway, a locally defined `TestEvent` JSON payload is utilized to replicate realistic incoming requests during testing. Additionally, for runtime observance and debugging in the deployed environment, Lambda function executions and errors are continually logged to dedicated AWS CloudWatch log groups.

3) Frontend: The React frontend is tested with Vitest [31], covering the helper functions and verifying that sample data is rendered correctly to the document.

4) Sensor Node Firmware: Firmware unit testing utilizes the AUnit framework [32] to cover modules like sensor readings, data aggregation, configurations, and Wi-Fi. Exhaustive MQTT mocking was omitted due to real-world deviations. Instead, the node subscribes to an MQTT test topic during operation to directly verify connection health and publish functionality.

B. Cost Estimation

The total costs for operating the system are split into AWS service costs and hardware costs.

1) AWS Service Costs: The figures in Table I reflect a monthly cost estimate based on an ESP-32 publishing one message every five seconds (roughly 518 000 messages per month). AWS IoT Core is the absolute dominant cost driver. Because each payload is evaluated against two IoT Core rules, AWS bills three units per publish, resulting in 1 554 000 billed message units. This accounts for over 75% of the total monthly cost (1.41 €).

Other minor expenses stem from Kinesis Data Firehose applying a 5 KB minimum billing unit—inflating the logical data volume to 2.59 GB (0.07 € per month with no free tier)—and Athena scanning costs for an estimated 660 history queries

Table I
COST ESTIMATION FOR AMAZON WEB SERVICES (AWS)

Service Details	Usage/Quantity	Free Tier	Post Free Tier
IoT Core [20]	518k msgs, 1554k billed	∼1.41 €	∼1.41 €
Lambda – handleTransformValidate [33]	518k inv, 128 MB / 200 ms	0.00 €	∼0.05 €
Lambda – handleLiveData [33]	228k inv, 128 MB / 200 ms	0.00 €	∼0.03 €
Lambda – handleHistoryData [33]	660 inv, 128 MB / 200 ms	0.00 €	∼0.00 €
DynamoDB [21]	518k WCU, 228k RCU, storage	∼0.01 €	∼0.07 €
S3 (Parquet + Query Results) [34]	∼0.5 GB stored	∼0.01 €	∼0.02 €
S3 (React Frontend) [34]	Minimal storage used	0.00 €	0.00 €
Firehose [22]	518k msgs, 2.59GB billed	∼0.07 €	∼0.07 €
Athena [35]	660 queries × 50 MB = 33 GB scanned	∼0.14 €	∼0.14 €
API Gateway [36]	229k requests	0.00 €	∼0.07 €
SNS [30]	Minimal alert volume	0.00 €	∼0.01 €
Total		**∼1.64 €**	**∼1.87 €**

per month. Storage, routing, and alert notification costs remain trivial.

The table contrasts the *Free Tier* expenditure of a new AWS account against the ongoing *Post Free Tier* cost once initial allowances expire. Overall, the architecture operates highly cost-efficiently, totalling approximately 1.64 € and 1.87 € per node per month, respectively.

2) Hardware Cost: The one-time hardware cost per sensor node is based on the components used for prototyping. A breakdown of the individual component costs is presented in Table II.

Table II
HARDWARE COST PER SENSOR NODE

Component	Unit Cost
ESP32-DevKitC	8 €
Bosch BME680	18 €
TSL2561 Light Sensor	5 €
DHT11 Sensor	3 €
Joy-it KY-026 Flame Sensor	3 €
Joy-it KY-028 Thermistor	3 €
Prototyping equipment	∼3 €
Total	**∼43 €**

C. Discussion

The evaluation reveals several architectural trade-offs regarding cost, latency, scalability, and security:

First, storing the API Gateway Access Key in the React frontend simplifies prototyping but presents a client-side security vulnerability. Production deployments necessitate a robust identity layer (e.g., AWS Cognito) to securely broker API transactions.

Second, the 5-second publishing interval ensures instantaneous anomaly alerting but drives up AWS IoT costs and edge power consumption (Table I). While reducing this frequency (e.g., to 30 seconds) would lower costs and allow the ESP32 to utilize Deep Sleep, this power-saving mode severs network availability and requires costly TLS handshakes upon waking. Consequently, strict real-time latency is structurally incompatible with extreme low-power sleep modes.

Third, hardcoding warning thresholds and notification recipients within Lambda functions minimizes complexity and prevents database read latencies during live execution. However, this static configuration lacks the flexibility needed for scalable, adjustable, multi-tenant laboratory deployments.

Finally, relying on absolute static thresholds fails to detect degrading air quality trends before critical hazards are breached [3]. Preemptive anomaly detection requires contextual pattern evaluation beyond rigid limits, such as machine learning analysis over historical data. Implementing machine learning for detecting anomalies (outliers) in live data instead of relying solely on thresholds would significantly enhance the outlier detection capabilities of the system for laboratory environments.

VIII. SUMMARY AND FUTURE WORK

Sensiq provides a scalable, end-to-end IoT solution for continuous laboratory environment monitoring. By combining a low-cost ESP32 sensor platform with a serverless AWS backend and a React dashboard, the system successfully delivers both low-latency alerting and robust historical analysis. It reliably monitors critical metrics including temperature, humidity, and air quality while maintaining a highly efficient operational footprint of under three U.S. dollars per node monthly.

As detailed in the evaluation (Section VII-C), future development prioritizes security, analytical maturity, and deployment flexibility. Planned operational improvements include migrating to AWS Cognito for robust authentication and enabling dynamic, per-device configuration of warning thresholds and notification recipients directly within the frontend. On the hardware level, integrating optimizing power consumption via deep-sleep configurations will broaden deployment capabilities. Lastly, introducing machine learning models will enhance the system with real-time outlier detection in the live data stream, providing preemptive anomaly alerts that overcome the limitations of static, threshold-based evaluation.

REFERENCES

[1] Claes Ahlneck and George Zografi. “The molecular basis of moisture effects on the physical and chemical stability of drugs in the solid state”. In: *International journal of pharmaceutics* 62.2-3 (1990), pp. 87–95.

[2] Norbert Kockmann, Philipp Thenée, Christoph Fleischer-Trebes, Gabriele Laudadio, and Timothy Noël. “Safety assessment in development and operation of modular continuous-flow processes”. In: *Reaction Chemistry & Engineering* 2.3 (2017), pp. 258–280.

[3] Rafia Mumtaz, Syed Mohammad Hassan Zaidi, Muhammad Zeeshan Shakir, Uferah Shafi, Muhammad Moeez Malik, Ayesha Haque, Sadaf Mumtaz, and Syed Ali Raza Zaidi. “Internet of things (Iot) based indoor air quality sensing and predictive analytic—A COVID-19 perspective”. In: *Electronics* 10.2 (2021), p. 184.

[4] Gonçalo Marques and Rui Pitarma. “An internet of things-based environmental quality management system to supervise the indoor laboratory conditions”. In: *Applied Sciences* 9.3 (2019), p. 438.

[5] Simon Völkl, Johannes Schieder, Sebastian Rosner, Andre Tien Vu, and Christoph P. Neumann. *Sensiq GitHub Repository*. [Online]. URL: https://github.com/s-voelkl/Sensiq.

[6] BerryBase. *ESP32 NodeMCU Development Board*. [Online]. URL: https://www.berrybase.de/en/esp32-nodemcu-development-board.

[7] Espressif Systems. *ESP32-WROOM-32 Datasheet*. [Online]. URL: https://documentation.espressif.com/esp32-wroom-32_datasheet_en.pdf.

[8] Asair. *DHT11 Temperature and Humidity Module*. [Online]. URL: https://asairsensors.com/product/dht11-sensor/.

[9] Joy-It. *KY-026 Analog Flame Sensor*. [Online]. URL: https://sensorkit.joy-it.net/en/sensors/ky-026.

[10] Joy-It. *KY-028 Temperature Sensor (Thermistor)*. [Online]. URL: https://sensorkit.joy-it.net/en/sensors/ky-028.

[11] BerryBase. *TSL2561 Digital Light / Brightness Sensor Datasheet*. [Online]. URL: https://www.berrybase.de/en/product-datasheet/019234a424aa7020aee21ad30f4ed8c8/create.

[12] Adafruit. *Adafruit TSL2561 Light Sensor Driver*. [Online]. URL: https://github.com/adafruit/Adafruit_TSL2561.

[13] Bosch Sensortec. *BME680 Gas, Pressure, Temperature and Humidity Sensor Datasheet*. [Online]. URL: https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme680-ds001.pdf.

[14] Random Nerd Tutorials. *ESP32: BME680 Environmental Sensor using Arduino IDE*. [Online]. URL: https://randomnerdtutorials.com/esp32-bme680-sensor-arduino/.

[15] Adafruit. *Adafruit BME680 Library*. [Online]. URL: https://github.com/adafruit/Adafruit_BME680.

[16] KiCad Services Corporation. *KiCad PCB Design Suite*. Version 10.0.3. 2026. URL: https://www.kicad.org.

[17] Arduino. *Arduino Language Reference*. [Online]. URL: https://www.arduino.cc/reference/en/.

[18] John S Steinhart and Stanley R Hart. “Calibration curves for thermistors”. In: *Deep sea research and oceanographic abstracts*. Vol. 15. 4. Elsevier. 1968, pp. 497–503.

[19] Amazon Web Services. *AWS Cloud Development Kit Documentation*. [Online]. URL: https://docs.aws.amazon.com/cdk/.

[20] Amazon. *AWS IoT Core Documentation*. [Online]. URL: https://docs.aws.amazon.com/iot/.

[21] Amazon. *Amazon DynamoDB Documentation*. [Online]. URL: https://docs.aws.amazon.com/dynamodb/.

[22] Amazon Web Services. *Amazon Kinesis Data Firehose Documentation*. [Online]. URL: https://docs.aws.amazon.com/firehose/.

[23] Amazon Web Services. *Amazon S3 Storage Classes*. [Online]. URL: https://aws.amazon.com/de/s3/storage-classes/.

[24] Amazon Web Services. *Managing the lifecycle of objects in Amazon S3*. [Online]. URL: https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lifecycle-mgmt.html.

[25] Amazon Web Services. *Control Access to Data Catalogs with IAM Policies in Amazon Athena*. [Online]. URL: https://docs.aws.amazon.com/athena/latest/ug/datacatalogs-iam-policy.html.

[26] Amazon Web Services. *Use execution parameters in Amazon Athena*. [Online]. URL: https://docs.aws.amazon.com/athena/latest/ug/querying-with-prepared-statements-querying-using-execution-parameters.html.

[27] Amazon Web Services. *Use Partition Projection with Amazon Athena*. [Online]. URL: https://docs.aws.amazon.com/athena/latest/ug/partition-projection.html.

[28] Amazon Web Services. *Use Workgroups to Control Query Access and Costs in Amazon Athena*. [Online]. URL: https://docs.aws.amazon.com/athena/latest/ug/workgroups-manage-queries-control-costs.html.

[29] Amazon Web Services. *Query Execution Status in Amazon Athena*. [Online]. URL: https://docs.aws.amazon.com/athena/latest/APIReference/API_QueryExecutionStatus.html.

[30] Amazon. *Amazon Simple Notification Service (SNS) Documentation*. [Online]. URL: https://docs.aws.amazon.com/sns/.

[31] Vitest. *Vitest Testing Framework*. [Online]. URL: https://vitest.dev/.

[32] Brian T. Park. *AUnit Testing Framework for Arduino*. Version 1.7.1. 2026. URL: https://github.com/bxparks/AUnit.

[33] Amazon. *AWS Lambda Documentation*. [Online]. URL: https://docs.aws.amazon.com/lambda/.

[34] Amazon. *Amazon Simple Storage Service (S3) Documentation*. [Online]. URL: https://docs.aws.amazon.com/s3/.

[35] Amazon. *Amazon Athena Documentation*. [Online]. URL: https://docs.aws.amazon.com/athena/.

[36] Amazon. *Amazon API Gateway Documentation*. [Online]. URL: https://docs.aws.amazon.com/apigateway/.